Assignment:- 4B

BCD to Excess-3 code converter using logic gates

1) Quick definition

- **BCD (Binary Coded Decimal)**: 4-bit code for decimal 0–9 (0000 to 1001).
- **Excess-3 (XS-3)**: code equal to (decimal value + 3) represented in 4-bit binary.
  So the converter performs: XS3 = BCD + 3 (binary add of constant 0011).

Use 4 input bits: B3 B2 B1 B0 (B3 = MSB).
Outputs: S3 S2 S1 S0 (S3 = MSB of Excess-3).

Valid inputs: 0000..1001 (0..9). Inputs 1010..1111 are invalid BCD — treat them as “don’t care” in simplification or show as invalid.

---

2) Truth table (only 0–9 shown)

(Decimal, BCD → Excess-3)

Dec B3 B2 B1 B0 -> S3 S2 S1 S0

0 0000 -> 0011 (3)

1 0001 -> 0100 (4)

2 0010 -> 0101 (5)

3 0011 -> 0110 (6)

4 0100 -> 0111 (7)

5 0101 -> 1000 (8)

6 0110 -> 1001 (9)

7 0111 -> 1010 (10)

8 1000 -> 1011 (11)

9 1001 -> 1100 (12)

(Inputs 1010–1111: invalid BCD.)

---

3) Minterms (for each output bit)

I used the valid BCD inputs (decimal 0..9) to find minterms where each S-bit = 1.

Let a = B3, b = B2, c = B1, d = B0. (So a = MSB.)

Minterms (decimal index = input B3B2B1B0 as binary → decimal):

- S3 (MSB) = 1 for inputs: 5,6,7,8,9
- S2 = 1 for inputs: 1,2,3,4,9
- S1 = 1 for inputs: 0,3,4,7,8
- S0 (LSB) = 1 for inputs: 0,2,4,6,8

---

4) Minimized boolean expressions (sum-of-products, ready for gates)

Using Karnaugh/simplification (valid inputs only), the simplified expressions are:

S3 (MSB)= (~a & b & c) OR (~a & b & d) OR (a & ~b & ~c)

S2 = (~a & ~b & c) OR (~b & ~c & d) OR (b & ~a & ~c & ~d)

S1 = (~a & c & d) OR (~a & ~c & ~d) OR (~b & ~c & ~d)

S0 (LSB)=(~a & ~d) OR (~b & ~c & ~d)

(Here ~ = NOT, & = AND, OR = OR.)

These forms are already compact and suitable for implementation with AND, OR, NOT gates. If you prefer, each OR clause can be implemented by a single OR gate combining the AND terms.

---

5) Gate-level implementation notes

- Implement NOT gates for a,b,c,d as needed.
- Build the AND product terms in the expressions above, then OR them to get each output bit.
- Example: S0 needs two product terms:
    - ~a & ~d (an AND with inverted a and d)
    - ~b & ~c & ~d (3-input AND with inverted inputs)

- OR those two outputs → S0.

Gate count (rough): depends on whether you use 2-input gates only or multi-input gates. Using multi-input AND/OR reduces gate count. The minimized forms above are efficient for manual gate implementation.

---

6) Simpler / practical implementation: use a 4-bit adder

Because XS3 = BCD + 0011, an easier hardware approach is to use a 4-bit ripple-carry adder:

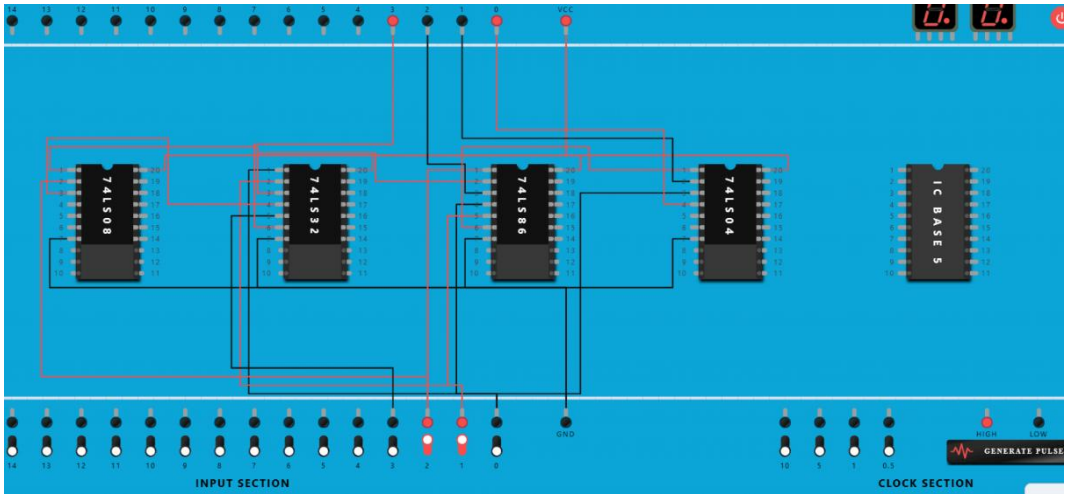
- Inputs: B3 B2 B1 B0
- Add constant: 0 0 1 1 (i.e., 0011)
- Connect to a 4-bit binary adder (use full adders chained).
- Advantage: easy to implement on breadboard/FPGA — only 4 full adders, no heavy boolean simplification.
- Example:
  - LSB sum S0 = B0 XOR 1 (i.e. ~B0)
  - Carry out of bit0 = B0 & 1 = B0
  - Proceed with bit1 adding B1 + 1 + carry0 (use full adder), etc. This approach is often preferred in labs because you can reuse standard full-adder ICs (e.g., 7483 or ripple adder modules).

---

7) Handling invalid BCD inputs

- For input patterns 1010–1111, the output is undefined as BCD is invalid. In a design you can:
  - Treat those as “don’t care” when simplifying (may yield simpler gates), **or**
  - Add a validity detector: Valid = ~(a & (b | c | d)) (i.e. Valid = 1 only for 0000..1001). If invalid, raise a flag or set outputs to some safe value.

A simple validity detect: Invalid = a & (b | c | d) (since any time a=1 and at least one lower bit is 1 → input >= 10).


VCC
74LS08
74LS32
74LS86
74LS04
IC BASE 5
GND
HIGH
LOW
GENERATE PULSE
INPUT SECTION
CLOCK SECTION